Solitaire

R.Nachtrieb

2024-11-19, revised 2025-07-19

Abstract

In practice, about one in ten solitaire games are won. But how many solitaire games *could be* won? We reproduce the solitaire rules in a computer program and "play" five hundred games. Following a simple strategy (pick the move with the biggest number of moves), we find about fourteen percent of the games can be won. Coding is in R, which is not fast but gets the job done.

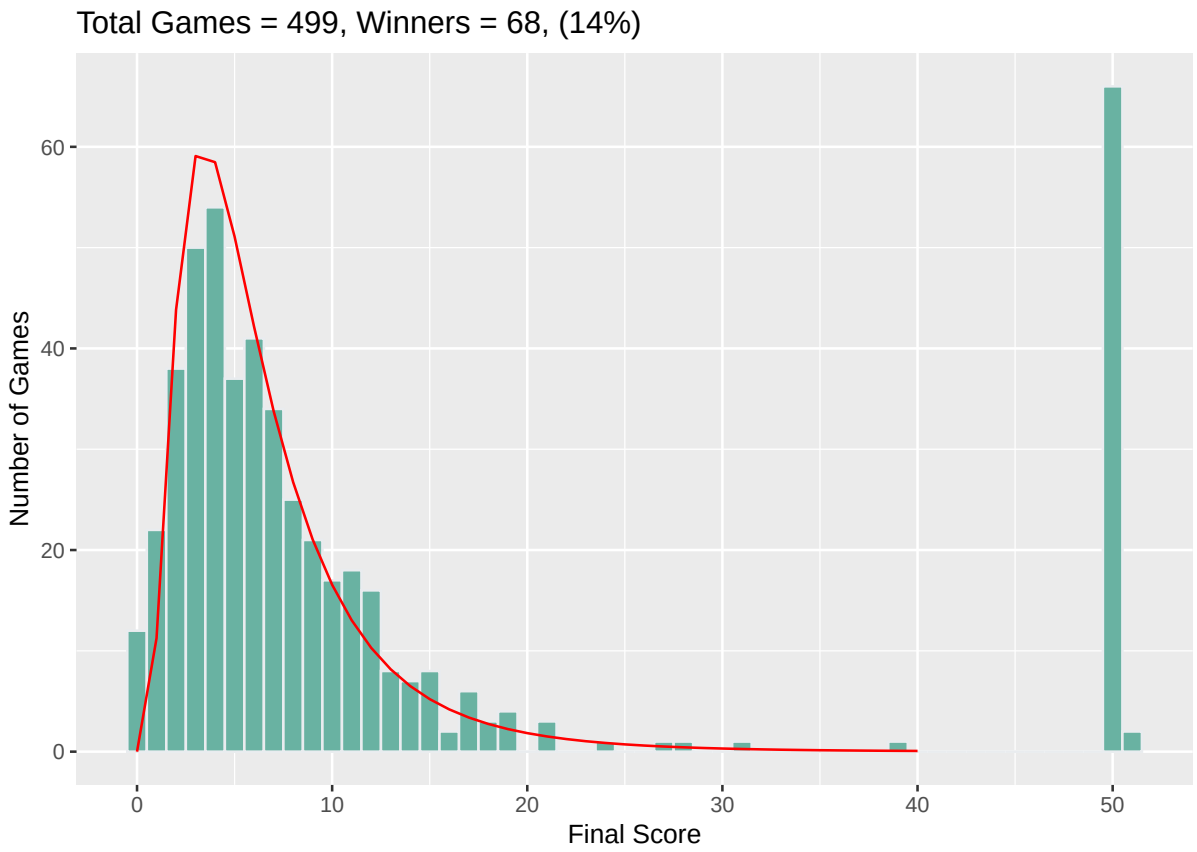


Figure 1: About 14% of solitaire games can be won (all the cards moved to the ace piles). For those games that cannot be won, the number of cards that can be moved to the ace piles is approximately log-normal distributed, with mean of 6.7 and standard deviation of 5.0.

Contents

1 Definitions **3**

2 Possible Moves **5**

3 Moving **6**
3.1 List of Possible Moves 6
3.2 Picking a Move 6
3.3 Sketch of a Strategy 6

A Math Background **8**

B Source Code **9**
B.1 play.r 9
B.2 get_deck.r 12
B.3 deal.r 13
B.4 load_get.r 14
B.5 sort_cols.r 15
B.6 get_move.r 16
B.7 make_move.r 20
B.8 one_step.r 23
B.9 make_plot.r 26

1 Definitions

- There are $N_c = 52$ *cards* in the deck. We can index the cards with $c = 1 \ldots 52$, and then make the face values and suits. (We index starting $c = 1$ so that we can use signed integers.)

- There are $N_i = 12$ *columns* where a card can be:

  1—4 The four *suit* columns (in order: C=clubs, D=diamonds, H=hearts, S=spades)

  5—11 The seven *pile* columns

  12 The *hand* column (with a pointer)

- Each column can have up to $N_j = 24$ cards. The occupied locations on a stack are contiguous.

- Each card has a *face* value. We'll assign a face id number to the face label: $A = 1$, $2 = 2$, $\ldots$, $9 = 9$, $T = 10$, $J = 11$, $Q = 12$, $K = 13$.

- Each card can be face-up or face-down. Without loss of generality, we'll call this the *sign*. For example, $(+1)$ indicates face up, and (-1) indicates face down.

- Every card must be in exactly one of the columns, at exactly one of the positions, with exactly one sign. No two cards can be in the same stack and position. A *state* is the stack number, position, and sign for every card.

  We typically describe the state as vector with 104 numbers: the position in the vector corresponds to the card id. The first 52 elements give the column number, the second 52 elements give the position and sign.

- To *sort* a state is to order the pile columns in a unique way, such that two states can be tested for equality. (Swapping two piles does not change any possible moves, thus the two states are equivalent.)

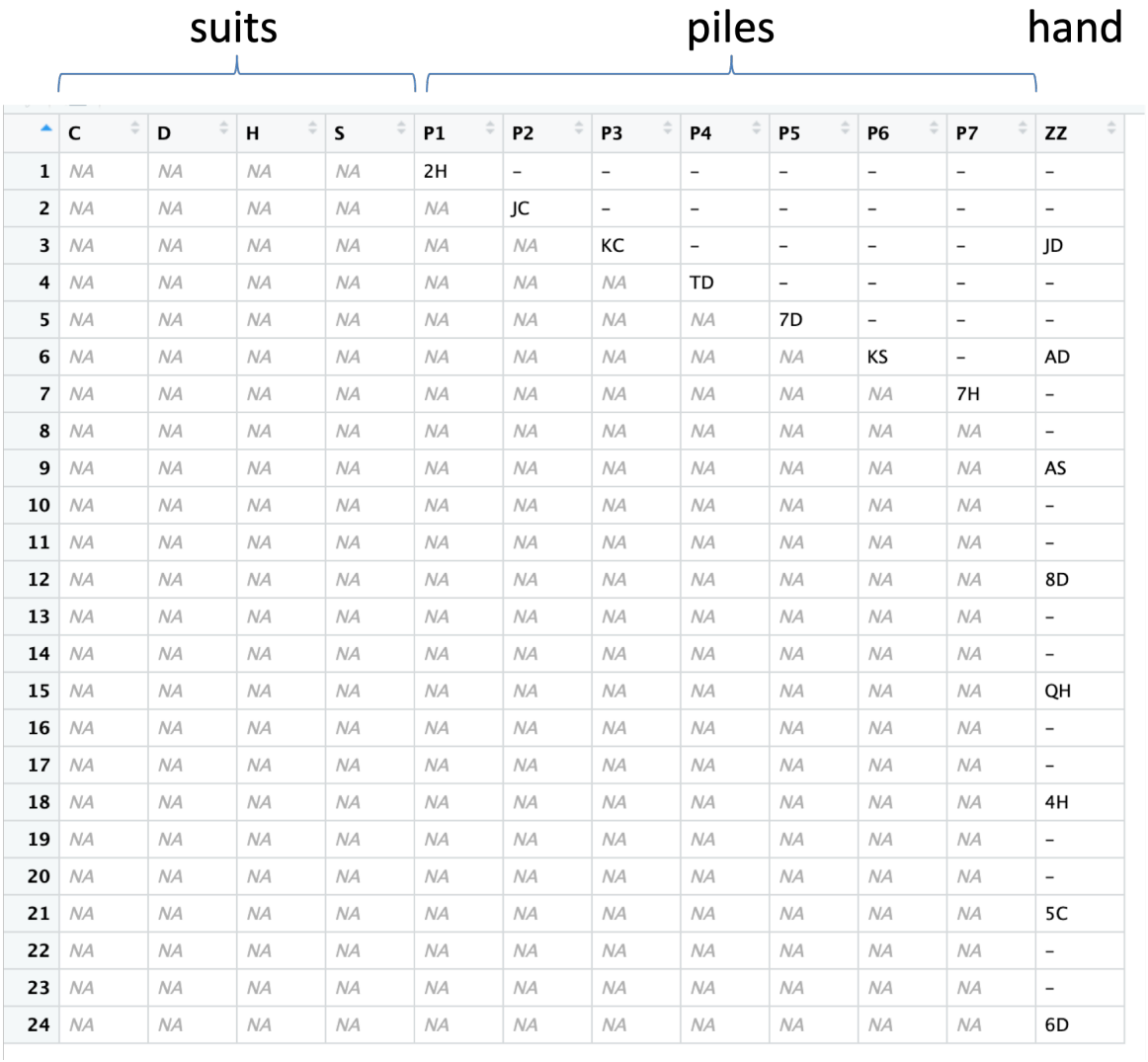
- A *state transition* is an allowed card *move*. A transtion changes an initial state to a new (different) state.

- The *final state* is when no further moves are possible.

- A *trajectory* is a single progression of states from initial to final.

- The *score* is the number of cards on the suit stacks in the final state.

- The *winning state* is the final state when all cards are in the proper sequence on the suit piles. The highest possible score is 52.



	C	D	H	S	P1	P2	P3	P4	P5	P6	P7	ZZ
1	NA	NA	NA	NA	2H	–	–	–	–	–	–	–
2	NA	NA	NA	NA	NA	JC	–	–	–	–	–	–
3	NA	NA	NA	NA	NA	NA	KC	–	–	–	–	JD
4	NA	NA	NA	NA	NA	NA	NA	TD	–	–	–	–
5	NA	NA	NA	NA	NA	NA	NA	NA	7D	–	–	–
6	NA	NA	NA	NA	NA	NA	NA	NA	NA	KS	–	AD
7	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	7H	–
8	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
9	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	AS
10	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
11	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
12	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	8D
13	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
14	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
15	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	QH
16	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
17	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
18	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	4H
19	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
20	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
21	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	5C
22	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
23	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	–
24	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	6D

Figure 2: Example initial state diagram from `seed=0`. Cards with "-" are face down. We can see six possible moves: From the hand to the suits: (1) AD to the D suit; (2) AS to the S suit. Between the piles: (1) TD on Pile 4 to JC on Pile 2. From the hand to the piles: (4) AS to the 2H in Pile 1; (5) QH to the KC in Pile 3. (6) QH to the KS in Pile 6. Because there are possible moves, this is not the final state.

2 Possible Moves

The following rules apply to moves:

- Only cards that are face up can move, either single cards or substacks (cards with alternating colors and descending rank).
- If a card is moved and a face-down card is underneath, the card turns face-up automatically (i.e. there is no move associated with turning a card face-up.) This is true for a pile or the hand.
- A state cannot be repeated.
- When playing solitaire by hand, moves are irreversible. (We'll waive this in our search for winnable games.)

The following moves are possible:

1. Move an ace card from the top of a pile or from the hand to one of the suits.
2. Move a non-ace card from the top of a pile or from the hand to one of the suits.
3. Move a king to an empty pile.
4. Move a card or a stack of cards to the top card of one of the piles.
5. Reset the hand. If we move a card from hand, then we have a different state. To reset the hand restores the every-third-up pattern in the hand

3 Moving

3.1 List of Possible Moves

Upon evaluation of a state we return a list of possible moves. Each move has a short description, the id of the card moving, the destination column, and the destination position.

If no moves are possible, then the game terminates. We suppress moves that lead to states previously-visited states.

3.2 Picking a Move

When playing the game by hand, picking a move is as simple as displaying a list of moves, then selecting the number of the corresponding move.

```
moves:
1 ace to suit AD.C
2 ace to suit AS.C
3 pile to pile TD.JC
4 hand to pile AS.2H
5 hand to pile QH.KC
6 hand to pile QH.KS
which option? (1.. 6 )
```

For each possible move, we can pre-calculate the what the new state would be. Once we have picked a move, to make the move is as easy as setting the state to the new state.

3.3 Sketch of a Strategy

We can describe an entire game with a directed graph, as below. Each state can be the *parent* state with several possible moves. Each move produces a new *child* state (and the process repeats).

Initially in a real solitaire game, every move leads to about three to give new moves. This geometric growth of the number of moves means we cannot realistically explore every possible move, even for a single game. So we adopt the following strategy: pick the move that leads to the largest number of moves. The rationale is:

- There is only one winning state: all cards on the suits.
- Every move of a card to the suits evolves our state closer to the winning state.
- If we don't run out of moves, we evolve closer to the winning state.
- We lose when we run out of moves.
- So pick the move that keeps us playing: the move with the biggest number of moves.

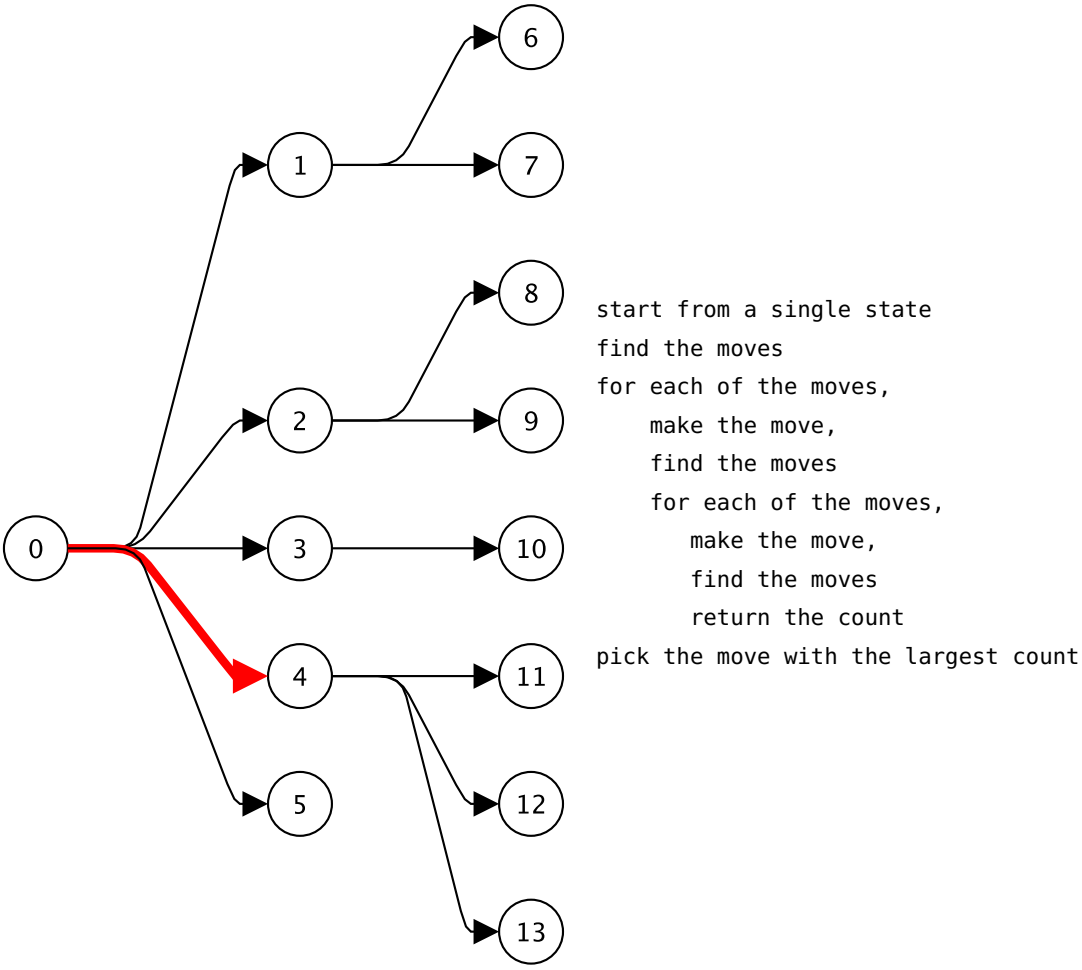


Figure 3: How moves are selected. In this example, State 0 has five possible moves. Each move is "tested". Move 5 is found to be a dead end. Move 3 gives one possible move. Moves 1 and 2 each give two moves. Move 4 gives three moves. The strategy heuristic is "play as long as possible", so we select move 4. The diagram shows testing ahead one generation, but the code actually tests ahead two generations.

A Math Background

How many initial top cards are there? How many initial states? 7 top cards in numbered stack + 8 top cards in hand stack = 15 cards. The number of initial hands is given by n choose k, where $n = 52$, $k = 15$, $n - k = 37$. Using Stirling's approximation, $\ln(z) \approx 153 - 26 - 97 = 31$, so $z \approx 10^{14}$.

Recall there are about $3 \times 10^7 \,\mathrm{s\,y^{-1}}$. If each initial condition took one second to process completely, it would take $10^7 \,\mathrm{y}$ to process all initial states. So we won't try every initial condition, only sample the space.

Here's the math behind n choose k:

$$z \equiv n\mathrm{C}k = \binom{n}{k} = \frac{n!}{k!\,(n-k)!} \tag{1}$$

Using Stirling's approximation[1]:

$$\ln(n!) \approx n\,\ln(n) - n \tag{2}$$

So

$$\ln(z) \approx \ln(n!) - \ln(k!) - \ln((n-k)!) \tag{3}$$

Logarithms in the natural base may be less intuitive than base ten. Recall ie $z = e^a = 10^b$, then $\ln z = a = b\,\ln 10$, so $b = \ln z / \ln(10)$ where $\ln 10 \approx 2.3$.

[1] https://en.wikipedia.org/wiki/Stirling%27s_approximation

B Source Code

B.1 play.r

```r
# solitaire
# R.Nachtrieb, 2025-06-08
# we store the state in two forms.
# 1. s (integer vector).  Most convenient for manipulation
#   vector of length [1:52,53:104].  First 52 are column of each card
#   second 52 are the postion*sign of each card
# 2. ch (string, comma separated variable).  Most convenient for storage
#   csv of s

library(tidyverse)
library(reshape2)

source("get_deck.r")
source("deal.r")
source("load_get.r")
source("sort_cols.r")
source("get_move.r")
source("make_move.r")
source("one_step.r")


play <- function(debug=0, s=NULL, seed=0){
  if (debug > 2) browser()

  if (is.null(s)) s <- deal(seed=seed)
  s <- sort_cols(s)

  gs <- NULL
  keep_going <- TRUE
  i=1
  while(keep_going) {
    this_gs <- data.frame(seed=seed,i=i,ch=get_ch_from_s(s))
    gs <- bind_rows(gs,this_gs)
    s2 <- one_step(s=s,debug=debug,gs=gs)
#    if (debug == -3) browser()

    keep_going <- (length(s2) == 104)
    if (keep_going) {
      cat(" suit score",sum(s2[1:52] < 5),"\n")
```

```
      s <- s2
      i <- i+1
    } else {
      cat(" FINAL suit score",sum(s[1:52] < 5),"\n")
      if (debug == -3) browser()
      return(gs)
    }
  }
}


sweep <- function(
    debug=0
    , seeds=seq(90,200)
    ,detail_rda = "gs_detail.rda"
    ,summary_rda = "gs_summary.rda"
) {
  gs_detail <- NULL
  gs_summary <- NULL

  for (this_seed in seeds){
    this_gs <- play(seed=this_seed)

    gs_detail <- bind_rows(gs_detail,this_gs)
    save(gs_detail,file=detail_rda)

    n <- dim(this_gs)[1]
    this_seed <- this_gs$seed[n]
    this_ch <- this_gs$ch[n]
    this_s <- get_s_from_ch(this_ch)
    this_score <- sum(this_s[1:52] < 5)
    this_summary <- data.frame(seed=this_seed,score=this_score)
    gs_summary <- bind_rows(gs_summary,this_summary)
    save(gs_summary,file=summary_rda)

    make_hist(gs_summary)
  }
}


make_hist <- function(gs_summary){
  n <- dim(gs_summary)[1]
```

```
  p <- ggplot(gs_summary,aes(x=score)) +
    geom_histogram(binwidth=2) +
    labs(title=paste("N=",n))
  print(p)
}
```

B.2 get_deck.r

```r
# set global variables that define a card deck

suit_label <- c("C","D","H","S")
face_label <- c("A","2","3","4","5","6","7","8","9","T","J","Q","K")

deck_id <- seq(1,52) # unique card id, AC..KS
deck_suit <- c(rep(1,13),rep(2,13),rep(3,13),rep(4,13)) # suits 1..4 correspond to C,D,H,S
deck_face <- rep(seq(1,13),4)
deck_color <- c(rep(0,13),rep(1,13),rep(1,13),rep(0,13))
deck_label <- paste0(face_label[deck_face],suit_label[deck_suit])

aces <- deck_id[deck_face == 1] # id of the aces
king <- deck_id[deck_face == 13] # id of the kings

piles <- seq(5,11) # the column numbers of the piles
```

B.3 deal.r

```
# make a new game. we can set the seed to reproduce results
deal <- function(seed=-1, debug=0){
  if (debug > 2) browser()
  if (!is.null(seed)) set.seed(seed)
  s0_col <- c(5 ,rep(6,2) ,rep(7,3) ,rep(8,4) ,rep(9,5) ,rep(10,6)
              ,rep(11,7) ,rep(12,24))
  s0_pos <- c(1 ,seq(1,2) ,seq(1,3) ,seq(1,4) ,seq(1,5) ,seq(1,6)
              ,seq(1,7)  ,seq(1,24))
  s0_face <- c(1 ,c(-1,1), c(rep(-1,2),1), c(rep(-1,3),1),
               c(rep(-1,4),1), c(rep(-1,5),1), c(rep(-1,6),1), c(rep(c(-1,-1,1),8)))
  n <- 52
  s_col <- rep(NA,n)
  s_possign <- rep(NA,n)
  u <- runif(n)
  if (seed == -1) u <- seq(1,n)/n
  r <- sort(u,index.return=TRUE)
  x <- r$ix
  s_col[x] <- s0_col
  s_possign[x] <- s0_pos*s0_face
  s <- c(s_col, s_possign)
  if (debug > 0) browser()
  return(s)
}
```

B.4 load_get.r

```r
# convert from s to ch, or vice versa.
# load ch from .rda

get_ch_from_s <- function(s){
  ch <- paste0(s,collapse=",")
  return(ch)
}

get_s_from_ch <- function(ch){
  s <- as.numeric(unlist(strsplit(ch,",")))
}

load_s <- function(debug=0, rda="gs.rda", n=NULL){
  if (debug > 2) browser()
  load(rda)
  if (is.null(n)) n <- dim(gs)[1]
  ch <- gs$ch[n]
  s <- get_s_from_ch(ch)
  if (debug > 2) browser()
  return(s)
}
```

B.5 sort_cols.r

```r
# sort the columns such that two states can be compared.
# need to sort only the piles, not suits not hand
# this means we change s_col, but not s_pos*s_sign
# any unique sort method is acceptable.
# want to sort in a way that is visually pleasing
# want to sort is a way that is relatively stable
# try increasing count, then id of top card
sort_cols <- function(s,debug=0){
  if (debug > 2) browser()

  s_col <- s[1:52]
  s_pos <- abs(s[53:104])
  s_sign <- sign(s[53:104])

  # find the pile count and top card in each pile
  pile_cnt <- rep(0,7) # starting pile count
  pile_top <- rep(0,7)
  for (i in 1:7) {
    reg_pile <- s_col==piles[i]
    pile_cnt[i] = sum(reg_pile)
    reg <- reg_pile & (s_pos==pile_cnt[i]) # find the top card
    if (sum(reg)>0) pile_top[i] <- deck_id[reg]
  }
  # value; this is what we'll sort
  val <- pile_cnt*53 + pile_top
  val2 <- sort(val, index.return=TRUE) # x$ix is all we care about

  if (debug > 1) browser()

  piles2 <- val2$ix+4
  s2 <- s
  for (i in 1:7) {
    reg_pile <- s_col==piles2[i]
    if (sum(reg_pile)>0) s2[1:52][reg_pile] <- piles[i]
  }
  if (debug > 0) browser()
  return(s2)
}
```

B.6 get_move.r

```
##############################################
# Finding the moves
# There are five types of move we'll look for:
#     description       move count
# a.    to suit         single
# b.    king to empty     single
# c.    pile to pile      multi
# d.    hand to pile      single
##############################################
get_move <- function(s, debug=0){
  if (debug > 2) browser()

  # basic definitions
  s_col <- s[1:52]
  s_pos <- abs(s[53:104])
  s_sign <- sign(s[53:104])

  # find the cards in the hand
  reg_hand <- (s_col == 12)
  hand <- deck_id[reg_hand] # id of the hand cards

  # find the face-up cards in the hand
  reg_hand_up <- (s_col == 12) & (s_sign>0)
  hand_up <- deck_id[reg_hand_up] # id of the hand up cards

  # find the face-up cards in the pile
  reg_pile_up <- (s_col > 4) & (s_col < 12) & (s_sign>0)
  pile_up <- deck_id[reg_pile_up] # id of the pile up cards

  # find the pile count and top card in each pile
  cnt <- rep(0,7) # starting pile count
  pile_top <- rep(NA,7)
  for (i in 1:7) {
    cnt[i] = sum(s_col == (i+4))
    reg <- (s_col == (i+4)) & (s_pos==cnt[i]) # find the top card
    if (sum(reg)>0) pile_top[i] <- deck_id[reg]
  }
  reg_pile_top <- !is.na(pile_top)
  pile_top <- pile_top[reg_pile_top] # id of the top pile cards
  n_top <- length(pile_top)

```

```
42
43  if (debug > 2) browser()
44
45  #####
46  # Now get the moves
47  #####
48  move <- data.frame()
49  m <- 0
50
51  ## define move to reset the hand.  This is an option
52  # if the face cards in the hand have changed from nominal
53
54  # find all the cards in the hand
55  hand_pos <- s_pos[hand]
56
57  x <- sort(hand_pos,index.return=TRUE)
58  ii <- x$ix
59
60  # find the correct hand pattern
61  n <- sum(reg_hand) # number of cards in hand
62  m <- as.integer(n/3)
63  r <- n - m*3
64  pat3 <- c(-1,-1,1)
65  pat_m <- rep(pat3,m)
66  pat_r <- numeric()
67  if (r > 0) pat_r <- pat3[seq(1,r)]
68  pat <- c(pat_m,pat_r)
69  if (n > 0) pat[n] <- 1
70
71  already_reset <- s_sign[hand[ii]] == pat
72  if (!all(already_reset)) {
73    pos_hand <- s_pos[hand_up]
74    reg <- pos_hand == max(pos_hand)
75    from_id <- hand_up[reg]
76    to_id <- from_id
77    to_col <- s[to_id]
78    to_pos <- s[to_id + 52]
79    this_move <- data.frame(kind = "z. reset hand"
80      ,from_id = from_id,to_col = to_col,to_pos = to_pos)
81    m <- m+1
82    move <- bind_rows(move,this_move)
83  }
```

```
84
85  if (debug > 2) browser()
86
87  ## move the top pile or hand to suits (ace or not)
88  all_from_id <- c(pile_top,hand_up)
89  n <- length(all_from_id)
90  if (n > 0){
91    for (i in 1:n){
92      from_id <- all_from_id[i]
93      to_id <- from_id-1 # provisional
94      to_col <- deck_suit[from_id]
95      to_pos <- deck_face[from_id]
96      if (from_id %in% aces) {
97        this_move <- data.frame(kind = "a. ace to suit"
98          ,from_id = from_id,to_col = to_col,to_pos = to_pos)
99        m <- m+1
100       move <- bind_rows(move,this_move)
101     } else if (!(from_id %in% aces) & (s[to_id] < 5)) {
102       this_move <- data.frame(kind = "b. non-ace to suit"
103         ,from_id = from_id,to_col = to_col,to_pos = to_pos)
104       m <- m+1
105       move <- bind_rows(move,this_move)
106     }
107   }
108 }
109
110 if (debug > 2) browser()
111
112 ## move king to an empty pile
113 reg_open <- cnt == 0
114 n_open <- sum(reg_open) # the count of empty columns
115 reg_king <-
116   ((s_col[king] > 4) & (s_col[king] < 12) & (s_pos[king]>1) & (s_sign[king]>0)) |
117   ((s_col[king] == 12) & (s_sign[king]>0))
118 n_king <- sum(reg_king) # the count of up kings not in suits
119 if ((n_open > 0) & (n_king > 0)) {
120   all_from_id <- king[reg_king]
121   to_cols <- piles[reg_open]
122   for (i in 1:n_king) {
123     for (j in 1:n_open) {
124       from_id <- all_from_id[i]
125       to_col <- to_cols[j]
```

```
        to_pos <- 1
        this_move <- data.frame(kind = "c. king to empty"
          ,from_id = from_id,to_col = to_col,to_pos = to_pos)
        m <- m+1
        move <- bind_rows(move,this_move)
      }
    }
  }

  if (debug > 2) browser()

  all_up <- c(pile_up,hand_up)
  # move to top pile
  if (n_top > 0) {
    for (i in 1:n_top) {
      to_id <- pile_top[i]
      to_col <- s[to_id]
      to_pos <- s[to_id+52]+1
      reg_from <-
        (deck_face[all_up] == (deck_face[to_id]-1)) &
        (deck_color[all_up] == (1-deck_color[to_id])) &
        (s[all_up] != to_col)
      n_from <- sum(reg_from)
      if (n_from > 0) {
        all_from_id <- all_up[reg_from]
        for (j in 1:n_from) {
          from_id <- all_from_id[j]
          this_move <- data.frame(kind = "d. to pile"
            ,from_id = from_id,to_col = to_col,to_pos = to_pos)
          m <- m+1
          move <- bind_rows(move,this_move)
        }
      }
    }
  }

  if (debug > 0) browser()
  return(move)
}
```

B.7 make_move.r

```
##############################################
# Making the move
# given single (1) anchor FROM and TO.
# 1. Find all (1+) card_id of FROM
# 2. Find all (1+) col and pos of TO
# 3. Change all (1+) col and pos of FROM to TO
# If FROM col is hand, then
### Treat Hand
# 4. Find all (0+) card_id in hand after FROM; pos--
# 5. Find all (0+) card_id in hand before FROM pos; face down. Card just below FROM pos face
    up
### Reset Hand
# 6. Turn all face down
# 7. Turn every third and last face up.
#
# description       move count  treat hand?
# -----------     ----------  -----------
# pile to suit    single      N
# pile to pile    multi       N
# king to empty   single      ? (Y if from hand)
# hand to suit    single      Y
# hand to pile    single      Y
#
##############################################
make_move <- function(s, from_id, to_col, to_pos, debug=0){
  if (debug > 2) browser()

  s_col <- s[1:52]
  s_pos <- abs(s[53:104])
  s_sign <- sign(s[53:104])

  # set the initial state
  s2_col <- s_col
  s2_pos <- s_pos
  s2_sign <- s_sign

  from_col <- s_col[from_id]
  from_pos <- s_pos[from_id]

  # by default, only the anchor card moves
  from_ids <- from_id
```

```
  to_cols <- to_col
  to_poss <- to_pos

  if ((from_col < 12) & (to_col > 4)) {
    reg <- (s_col == from_col) & (s_pos >= from_pos)
    from_ids <- deck_id[reg]
    to_cols <- rep(to_col,sum(reg))
    to_poss <- to_pos + s_pos[reg] - s_pos[from_id]
  }

  # now make the changes!
  s2_col[from_ids] <- to_cols
  s2_pos[from_ids] <- to_poss
  s2_sign[from_ids] <- 1 # should already have been up

  # Now turn the cards up
  if ((from_col < 12) & (from_pos > 1)) { # from pile
    reg <- (s_col == from_col) & (s_pos == from_pos-1)
    up_ids <- deck_id[reg]
    s2_sign[up_ids] <- 1
  } else if (from_col == 12) { # from hand
    reg <- (s_col == 12) & (s_pos < from_pos)
    if (sum(reg) > 0) {
      before_ids <- deck_id[reg]
      s2_sign[before_ids] <- -1
    }
    reg <- (s_col == 12) & (s_pos == from_pos-1)
    if (sum(reg) > 0) {
      below_ids <- deck_id[reg]
      s2_sign[below_ids] <- 1
    }
    reg <- (s_col == 12) & (s_pos > from_pos)
    if (sum(reg) > 0) {
      after_ids <- deck_id[reg]
      s2_pos[after_ids] <- s2_pos[after_ids] - 1 # shift down
    }
  }

  # if reset the hand, change just face not position or column
  if ((from_col==to_col) & (from_pos==to_pos)) {

    # find all the cards in the hand
```

```
    reg_hand <- (s2_col == 12)
    hand_id <- deck_id[reg_hand] # id of the hand up cards; sorted!
    hand_pos <- s2_pos[reg_hand]

    x <- sort(hand_pos,index.return=TRUE)

    # now make the changes!
    n <- sum(reg_hand) # number of cards in hand
    m <- as.integer(n/3)
    r <- n - m*3
    pat3 <- c(-1,-1,1)
    pat_m <- rep(pat3,m)
    pat_r <- numeric()
    if (r > 0) pat_r <- pat3[seq(1,r)]
    pat <- c(pat_m,pat_r)
    if (n > 0) pat[n] <- 1

    s2_sign[hand_id[x$ix]] <- pat
  }

  s2 <- c(s2_col,s2_pos*s2_sign)
  if (debug > 0) browser()
  return(s2)
}
```

B.8 one_step.r

```
################################################################
# we start from a single state
# find the moves
# for each of the moves,
#    make the move,
#    find the moves
#    for each of the moves,
#       make the move,
#       find the moves
#       return the count
################################################################

one_step <- function(debug=0, s=NULL, seed=0, gs=gs){
  if (debug > 2) browser()

  if (is.null(s)) s <- deal(seed=seed)
  s0 <- sort_cols(s)
  df <- NULL

  # find the moves
  move1 <- get_move(s0, debug=debug)
  N1 <- dim(move1)[1]
  if (N1 > 0) {
    for (i1 in 1:N1) {
      # make the move
      from_id <- move1$from_id[i1]
      to_col <- move1$to_col[i1]
      to_pos <- move1$to_pos[i1]
      s1 <- make_move(s0,from_id,to_col,to_pos,debug=debug)
      s1 <- sort_cols(s1, debug=debug)
      ch1 <- get_ch_from_s(s1)

      N2<-0
      N3<-0

      # find the moves
      move2 <- get_move(s1, debug=debug)
      N2 <- dim(move2)[1]

      if (N2 > 0) {
        for (i2 in 1:N2) {
```

```
42
43          # make the move
44          from_id <- move2$from_id[i2]
45          to_col <- move2$to_col[i2]
46          to_pos <- move2$to_pos[i2]
47          s2 <- make_move(s1,from_id,to_col,to_pos,debug=debug)
48          s2 <- sort_cols(s2)
49          # find the moves
50          move3 <- get_move(s2, debug=debug)
51          N3 <- dim(move3)[1]
52
53          this_df <- data.frame(
54            i1=i1,ch1=ch1,i2=i2,N1=N1,N2=N2,N3=N3
55          )
56          df <- bind_rows(df,this_df)
57          if (debug == -1) browser()
58        }
59      }
60    }
61  }
62  mm0 <- df %>%
63    mutate(N23=N2*N3) %>%
64    filter(!(ch1 %in% gs$ch))
65
66
67  if (dim(mm0)[1] > 0) {
68    mm <- mm0 %>%
69      group_by(i1) %>%
70      summarize(score = mean(N23)) %>%
71      ungroup() %>%
72      filter(score == max(score))
73
74    this_score <- mm$score[1]
75    cat("move score",this_score)
76
77    if (this_score > 0) {
78      i <- mm$i1[1]
79      from_id <- move1$from_id[i]
80      to_col <- move1$to_col[i]
81      to_pos <- move1$to_pos[i]
82      s <- make_move(s0,from_id,to_col,to_pos,debug=debug)
83      s <- sort_cols(s)
```

```
      if (debug == -2) browser()
      return(s)
    } else {
      return(numeric())
    }

  } else {
    return(numeric())
  }

}
```

B.9 make_plot.r

```
library(tidyverse)

agg_gs <- function(
    debug=0
    ,all_summary_rda = c(
      "gs_summary_0_89.rda"
      ,"gs_summary_90_200.rda"
      ,"gs_summary_201_221.rda"
      ,"gs_summary_223_300.rda"
      ,"gs_summary_301_316.rda"
      ,"gs_summary_318_400.rda"
      ,"gs_summary_401_500.rda"
    )
    ,all_detail_rda = c(
      "gs_detail_0_89.rda"
      ,"gs_detail_90_200.rda"
      ,"gs_detail_201_221.rda"
      ,"gs_detail_223_300.rda"
      ,"gs_detail_301_316.rda"
      ,"gs_detail_318_400.rda"
      ,"gs_detail_401_500.rda"
    )
){
  if (debug > 2) browser()

  all_gs_detail <- NULL
  for (this_rda in all_detail_rda) {
    load(this_rda)
    all_gs_detail <- bind_rows(all_gs_detail,gs_detail)
  }
  save(all_gs_detail,file="gs_detail.rda")

  all_gs_summary <- NULL
  for (this_rda in all_summary_rda) {
    load(this_rda)
    all_gs_summary <- bind_rows(all_gs_summary,gs_summary)
  }
  save(all_gs_summary,file="gs_summary.rda")

  return(all_gs_summary)
}
```

```r

make_plot <- function(
    gs_summary
    ,plot=FALSE
    ,debug=0
    ,binwidth=2
){
  if(debug > 2) browser()

  xmax <- 40

  mm1 <- gs_summary %>%
    filter(score <= xmax) %>%
    summarize(
      games = n()
      ,tot = sum(score)
      ,mX = mean(score)
      ,vX = var(score)
    )

  xx <- seq(0*binwidth/2,xmax,binwidth)

  mX <- mm1$mX
  vX <- mm1$vX
  sX <- sqrt(vX)

  ml <- log(mX*mX/sqrt(mX*mX+vX))
  vl <- log(1+vX/(mX*mX))
  sl <- sqrt(vl)


  nsum <- mm1$games
  #yy <- dpois(xx,mu)*ntot*binwidth
  yy <- dlnorm(xx,meanlog=ml,sdlog=sl)*nsum*binwidth
  mm <- data.frame(x=xx,y=yy)




  ntot <- dim(gs_summary)[1]

```

```
  p <- ggplot(gs_summary,aes(x=score)) +
    geom_histogram(binwidth=binwidth,fill='#69b2a2',color='#e9ecef' ) +
    labs(x="Final Score",y="Number of Games"
         ,title=paste0("Total Games = ",ntot,", Winners = ",ntot-nsum," (",round(100*(1-nsum/
             ntot),0),"%)")) +
    geom_line(data=mm,aes(x=x,y=y), color="red")



  if (plot) pdf(file="gs_summary.pdf",width=7,height=5)
  print(p)
  if (plot) dev.off()

  if(debug > 0) browser()
}
```